

ESP-32

```
#include "esp_camera.h"

#include <WiFi.h>

#include <WebServer.h>

const char* ssid = "FireBot";

const char* password = "";

WebServer server(80);

// --- Motor Pins ---

#define IN1 14

#define IN2 15

#define IN3 13

#define IN4 12

// --- PWM Channels (Core 2.x Compatible) ---

#define CH1 0

#define CH2 1

#define CH3 2

#define CH4 3

// --- Global Variables ---

int robotSpeed = 255;

char currentMode = 'M';

bool isEmergency = false;

unsigned long lastCmdTime = 0; // Fail-safe tracker

// --- Ai-Thinker ESP32-CAM Pins ---

#define PWDN_GPIO_NUM 32

#define RESET_GPIO_NUM -1
```

```
#define XCLK_GPIO_NUM 0
#define SIOD_GPIO_NUM 26
#define SIOC_GPIO_NUM 27
#define Y9_GPIO_NUM 35
#define Y8_GPIO_NUM 34
#define Y7_GPIO_NUM 39
#define Y6_GPIO_NUM 36
#define Y5_GPIO_NUM 21
#define Y4_GPIO_NUM 19
#define Y3_GPIO_NUM 18
#define Y2_GPIO_NUM 5
#define VSYNC_GPIO_NUM 25
#define HREF_GPIO_NUM 23
#define PCLK_GPIO_NUM 22

// --- HTML UI ---

const char INDEX_HTML[] PROGMEM = R"rawliteral(
<!DOCTYPE html>

<html>

<head>

  <title>FireBot Pro</title>

  <meta name="viewport" content="width=device-width, initial-scale=1, maximum-
scale=1, user-scalable=no">

  <style>

    body { font-family: Arial, sans-serif; text-align: center; background-color: #2c3e50; color:
white; margin: 0; padding: 10px; }

    img { width: 100%; max-width: 600px; border: 4px solid #34495e; border-radius: 10px;
margin-bottom: 10px; }
```

```
.tabs { display: flex; justify-content: center; gap: 5px; max-width: 340px; margin: 0 auto 15px auto; flex-wrap: wrap; }
```

```
.tab-btn { flex: 1; padding: 10px 5px; font-size: 12px; font-weight: bold; border: none; border-radius: 5px; cursor: pointer; color: white; background-color: #7f8c8d; min-width: 70px; }
```

```
.control-section { display: none; }
```

```
.active-section { display: block; }
```

```
.pad { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; max-width: 340px; margin: 0 auto; }
```

```
.btn { background-color: #3498db; color: white; border: none; padding: 15px 5px; font-size: 14px; font-weight: bold; border-radius: 10px; cursor: pointer; }
```

```
.btn-red { background-color: #e74c3c; }
```

```
.speed-control { margin-top: 20px; display: flex; justify-content: center; align-items: center; gap: 15px; }
```

```
.speed-text { font-size: 16px; font-weight: bold; }
```

```
.btn-speed { background-color: #f39c12; color: white; border: none; padding: 10px 15px; font-size: 12px; font-weight: bold; border-radius: 5px; }
```

```
.status-box { padding: 20px; background-color: #34495e; border-radius: 10px; max-width: 320px; margin: 0 auto; }
```

```
#emergencyOverlay {
```

```
display: none; position: fixed; top: 0; left: 0; width: 100%; height: 100%;
```

```
background: rgba(192, 57, 43, 0.95); z-index: 9999; flex-direction: column;
```

```
justify-content: center; align-items: center;
```

```
}
```

```
.resume-btn {
```

```
background-color: #2ecc71; color: white; font-size: 24px; font-weight: bold;
```

```
padding: 20px 40px; border: none; border-radius: 10px; cursor: pointer; margin-top: 20px;
```

```
}
```

```
</style>

</head>

<body onload="switchTab('M')">

  <div id="emergencyOverlay">

    <h1 style="font-size: 40px; margin: 0;">EMERGENCY STOPPED!</h1>

    <p style="font-size: 18px;">All systems locked.</p>

    <button class="resume-btn" onclick="toggleEmergency(0)">RESUME SYSTEM</button>

  </div>

  <h2>FIREBOT SYSTEM</h2>

  

  <div class="tabs">

    <button id="tabManual" class="tab-btn" onclick="switchTab('M')">MANUAL</button>

    <button id="tabAuto" class="tab-btn" onclick="switchTab('A')">LINE</button>

    <button id="tabFire" class="tab-btn" onclick="switchTab('F')">FIRE</button>

    <button id="tabYolo" class="tab-btn" onclick="switchTab('Y')">YOLO AI</button>

  </div>

  <div id="manualControls" class="control-section">

    <div class="pad">

      <div></div>

      <button class="btn" onmousedown="sendCmd('F')" onmouseup="sendCmd('S')"  
ontouchstart="sendCmd('F')" ontouchend="sendCmd('S')">FORWARD</button>

      <div></div>

      <button class="btn" onmousedown="sendCmd('L')" onmouseup="sendCmd('S')"  
ontouchstart="sendCmd('L')" ontouchend="sendCmd('S')">LEFT</button>

      <button class="btn btn-red" onclick="toggleEmergency(1)">STOP</button>

    </div>

  </div>

</body>

</html>
```

```
<button class="btn" onmousedown="sendCmd('R')" onmouseup="sendCmd('S')"  
ontouchstart="sendCmd('R')" ontouchend="sendCmd('S')">RIGHT</button>
```

```
<div></div>
```

```
<button class="btn" onmousedown="sendCmd('B')" onmouseup="sendCmd('S')"  
ontouchstart="sendCmd('B')" ontouchend="sendCmd('S')">BACKWARD</button>
```

```
<div></div>
```

```
</div>
```

```
<div class="speed-control">
```

```
<button class="btn-speed" onclick="changeSpeed(-10)">SPEED DOWN</button>
```

```
<div class="speed-text">SPEED: <span id="speedDisplay">255</span></div>
```

```
<button class="btn-speed" onclick="changeSpeed(10)">SPEED UP</button>
```

```
</div>
```

```
</div>
```

```
<div id="autoControls" class="control-section">
```

```
<div class="status-box">
```

```
<div style="color:#2ecc71;font-weight:bold;margin-bottom:15px;">AUTO MODE  
ACTIVE</div>
```

```
<button class="btn btn-red" style="width:100%"  
onclick="toggleEmergency(1)">EMERGENCY STOP</button>
```

```
</div>
```

```
</div>
```

```
<div id="fireControls" class="control-section">
```

```
<div class="status-box">
```

```
<div style="color:#e67e22;font-weight:bold;margin-bottom:15px;">FIREFIGHTER MODE  
ACTIVE</div>
```

```
<button class="btn btn-red" style="width:100%"  
onclick="toggleEmergency(1)">EMERGENCY STOP</button>
```

```
</div>
```

```

</div>

<div id="yoloControls" class="control-section">

  <div class="status-box">

    <div style="color:#9b59b6;font-weight:bold;margin-bottom:15px;">YOLO PYTHON
    CONTROL ACTIVE</div>

    <button class="btn btn-red" style="width:100%"
onclick="toggleEmergency(1)">EMERGENCY STOP</button>

  </div>

</div>

<script>

let currentSpeed = 255;

function switchTab(mode) {

  document.querySelectorAll('.control-section').forEach(s => s.classList.remove('active-
section'));

  document.querySelectorAll('.tab-btn').forEach(b => b.style.backgroundColor =
'#7f8c8d');

  if (mode === 'M') {

    document.getElementById('manualControls').classList.add('active-section');

    document.getElementById('tabManual').style.backgroundColor = '#e67e22';

  } else if (mode === 'A') {

    document.getElementById('autoControls').classList.add('active-section');

    document.getElementById('tabAuto').style.backgroundColor = '#27ae60';

  } else if (mode === 'F') {

    document.getElementById('fireControls').classList.add('active-section');

    document.getElementById('tabFire').style.backgroundColor = '#c0392b';

  } else if (mode === 'Y') {

    document.getElementById('yoloControls').classList.add('active-section');

```

```
    document.getElementById('tabYolo').style.backgroundColor = '#8e44ad';
  }
  fetch('/mode?val=' + mode);
}
function sendCmd(cmd) { fetch('/action?cmd=' + cmd); }
function changeSpeed(step) {
  currentSpeed = Math.min(255, Math.max(50, currentSpeed + step));
  document.getElementById('speedDisplay').innerText = currentSpeed;
  fetch('/speed?val=' + currentSpeed);
}
function toggleEmergency(state) {
  if (state === 1) {
    document.getElementById('emergencyOverlay').style.display = 'flex';
    fetch('/emergency?state=1');
  } else {
    document.getElementById('emergencyOverlay').style.display = 'none';
    fetch('/emergency?state=0');
  }
}
// Camera Load Optimization (300ms)
setInterval(function() {
  document.getElementById('camFeed').src = '/capture?r=' + Math.random();
}, 300);
</script>
</body>
</html>
```

```

)rawliteral";

// --- Motor Functions (Core 2.x Setup) ---

void setupMotors() {

    ledcSetup(CH1, 2000, 8);

    ledcSetup(CH2, 2000, 8);

    ledcSetup(CH3, 2000, 8);

    ledcSetup(CH4, 2000, 8);

    ledcAttachPin(IN1, CH1);

    ledcAttachPin(IN2, CH2);

    ledcAttachPin(IN3, CH3);

    ledcAttachPin(IN4, CH4);

    stopMotors();

}

void moveForward() { ledcWrite(CH1, robotSpeed); ledcWrite(CH2, 0); ledcWrite(CH3,
robotSpeed); ledcWrite(CH4, 0); }

void moveBackward() { ledcWrite(CH1, 0); ledcWrite(CH2, robotSpeed); ledcWrite(CH3, 0);
ledcWrite(CH4, robotSpeed); }

void turnRight() { ledcWrite(CH1, robotSpeed); ledcWrite(CH2, 0); ledcWrite(CH3, 0);
ledcWrite(CH4, robotSpeed); }

void turnLeft() { ledcWrite(CH1, 0); ledcWrite(CH2, robotSpeed); ledcWrite(CH3,
robotSpeed); ledcWrite(CH4, 0); }

void stopMotors() { ledcWrite(CH1, 0); ledcWrite(CH2, 0); ledcWrite(CH3, 0);
ledcWrite(CH4, 0); }

void executeCmd(char cmd) {

    if (isEmergency) {

        stopMotors();

        return;
    }
}

```

```

}

if (cmd == 'F') moveForward();

else if (cmd == 'B') moveBackward();

else if (cmd == 'L') turnLeft();

else if (cmd == 'R') turnRight();

else if (cmd == 'S') stopMotors();

}

// --- Web Endpoints ---

void handleRoot() { server.send(200, "text/html", INDEX_HTML); }

void handleCapture() {

    camera_fb_t* fb = esp_camera_fb_get();

    if (!fb) { server.send(500, "text/plain", "Cam Fail"); return; }

    server.send_P(200, "image/jpeg", (const char*)fb->buf, fb->len);

    esp_camera_fb_return(fb);

}

void handleCommand() {

    lastCmdTime = millis(); // Track when the last command was received

    if (server.hasArg("cmd")) {

        char cmd = server.arg("cmd")[0];

        if (currentMode == 'M' || currentMode == 'Y' || cmd == 'S') {

            executeCmd(cmd);

        }

    }

    server.send(200, "text/plain", "OK");

}

void handleMode() {

```

```

if (server.hasArg("val")) {
    currentMode = server.arg("val")[0];
    stopMotors();
    if (!isEmergency) {
        Serial.print("M:");
        Serial.println(currentMode);
    }
}

server.send(200, "text/plain", "OK");
}

void handleSpeed() {
    if (server.hasArg("val")) {
        int reqSpeed = server.arg("val").toInt();
        robotSpeed = (reqSpeed < 50) ? 50 : reqSpeed;
        if (robotSpeed > 255) robotSpeed = 255;
    }

    server.send(200, "text/plain", "OK");
}

void handleEmergency() {
    if (server.hasArg("state")) {
        if (server.arg("state") == "1") {
            isEmergency = true;
            stopMotors();
            Serial.print("M:");
            Serial.println('E');
        } else {

```

```

    isEmergency = false;

    Serial.print("M:");

    Serial.println(currentMode);
}
}

server.send(200, "text/plain", "OK");
}

void setup() {
    Serial.begin(9600);

    setupMotors();

    camera_config_t config;

    config.ledc_channel = LEDC_CHANNEL_0; config.ledc_timer = LEDC_TIMER_0;

    config.pin_d0=Y2_GPIO_NUM; config.pin_d1=Y3_GPIO_NUM;
    config.pin_d2=Y4_GPIO_NUM; config.pin_d3=Y5_GPIO_NUM;

    config.pin_d4=Y6_GPIO_NUM; config.pin_d5=Y7_GPIO_NUM;
    config.pin_d6=Y8_GPIO_NUM; config.pin_d7=Y9_GPIO_NUM;

    config.pin_xclk=XCLK_GPIO_NUM; config.pin_pclk=PCLK_GPIO_NUM;
    config.pin_vsync=VSYNC_GPIO_NUM; config.pin_href=HREF_GPIO_NUM;

    config.pin_sccb_sda=SIOD_GPIO_NUM; config.pin_sccb_scl=SIOD_GPIO_NUM;
    config.pin_pwdn=PWDN_GPIO_NUM; config.pin_reset=RESET_GPIO_NUM;

    config.xclk_freq_hz=20000000; config.pixel_format=PIXFORMAT_JPEG;
    config.frame_size=FRAMESIZE_VGA; config.jpeg_quality=12; config.fb_count=1;

    esp_camera_init(&config);

    WiFi.softAP(ssid, password);

    server.on("/", handleRoot);

    server.on("/capture", handleCapture);

    server.on("/action", handleCommand);
}

```

```

server.on("/mode", handleMode);

server.on("/speed", handleSpeed);

server.on("/emergency", handleEmergency);

server.begin();
}

void loop() {

  server.handleClient();

  // Fail-safe logic: Only auto-stop if in Manual or YOLO mode and Wi-Fi drops for 1 sec
  if ((currentMode == 'M' || currentMode == 'Y') && (millis() - lastCmdTime > 1000)) {

    stopMotors();

  }

  // Handle Arduino Serial Feedback

  if ((currentMode == 'A' || currentMode == 'F') && Serial.available()) {

    char autoCmd = Serial.read();

    executeCmd(autoCmd);

  }

}

```

Arduino UNO

```

#include <SoftwareSerial.h>

SoftwareSerial ESPSerial(10, 11);

#define LEFT_SENSOR 4

#define RIGHT_SENSOR 5

#define FLAME_L 6

#define FLAME_F 7

#define FLAME_R 8

```

```
#define BUZZER 3

char sysMode = 'M';

// Non-blocking Variables

unsigned long previousLineTime = 0;

unsigned long previousFireSpamTime = 0;

// --- GLOBAL FIRE LOCK VARIABLE ---

bool fireAlarmLocked = false;

void setup() {

  Serial.begin(9600);

  ESPSerial.begin(9600);

  pinMode(LED_BUILTIN, OUTPUT);

  pinMode(LEFT_SENSOR, INPUT);

  pinMode(RIGHT_SENSOR, INPUT);

  pinMode(FLAME_L, INPUT_PULLUP);

  pinMode(FLAME_F, INPUT_PULLUP);

  pinMode(FLAME_R, INPUT_PULLUP);

  pinMode(BUZZER, OUTPUT);

  noTone(BUZZER);

}

void loop() {

  unsigned long currentMillis = millis();

  while (ESPSerial.available() > 0) {

    char c = ESPSerial.read();

    if (c == 'M') {

      delay(10);

      if (ESPSerial.available() >= 2) {
```

```

char colon = ESPSerial.read();

char modeChar = ESPSerial.read();

if (colon == ':') {

    if (modeChar == 'A' || modeChar == 'F' || modeChar == 'M' || modeChar == 'E') {

        sysMode = modeChar;

        if (sysMode == 'E' || sysMode == 'M') {

            fireAlarmLocked = false;

            noTone(BUZZER);

            digitalWrite(LED_BUILTIN, LOW);

        }

    }

}

}

}

}

}

if (sysMode == 'A') {

    if (currentMillis - previousLineTime >= 100) {

        int leftValue = digitalRead(LEFT_SENSOR);

        int rightValue = digitalRead(RIGHT_SENSOR);

        if (leftValue == 0 && rightValue == 0) ESPSerial.print('F');

        else if (leftValue == 1 && rightValue == 0) ESPSerial.print('R');

        else if (leftValue == 0 && rightValue == 1) ESPSerial.print('L');

        else ESPSerial.print('S');

        previousLineTime = currentMillis;

    }

}

```

```

else if (sysMode == 'F') {
    static int patrolPhase = 0;
    static unsigned long phaseTimer = 0;
    static unsigned long previousRadarTime = 0;
    if (!fireAlarmLocked) {
        int fL = digitalRead(FLAME_L);
        int fF = digitalRead(FLAME_F);
        int fR = digitalRead(FLAME_R);
        if (fL == 0 || fF == 0 || fR == 0) {
            fireAlarmLocked = true;
        }
    }
    if (fireAlarmLocked) {
        digitalWrite(LED_BUILTIN, HIGH);
        tone(BUZZER, 1000);
        if (currentMillis - previousFireSpamTime >= 100) {
            ESPSerial.print('S');
            previousFireSpamTime = currentMillis;
        }
    }
    else {
        digitalWrite(LED_BUILTIN, LOW);
        noTone(BUZZER);
        if (patrolPhase == 0) {
            if (currentMillis - previousRadarTime >= 100) {
                ESPSerial.print('F');
            }
        }
    }
}

```

```
    previousRadarTime = currentMillis;
}
if (currentMillis - phaseTimer >= 4000) {
    patrolPhase = 1; phaseTimer = currentMillis;
}
}
else if (patrolPhase == 1) {
    if (currentMillis - previousRadarTime >= 100) {
        ESPSerial.print('L');
        previousRadarTime = currentMillis;
    }
    if (currentMillis - phaseTimer >= 150) {
        patrolPhase = 2; phaseTimer = currentMillis;
    }
}
else if (patrolPhase == 2) {
    if (currentMillis - previousRadarTime >= 100) {
        ESPSerial.print('R');
        previousRadarTime = currentMillis;
    }
    if (currentMillis - phaseTimer >= 300) {
        patrolPhase = 3; phaseTimer = currentMillis;
    }
}
else if (patrolPhase == 3) {
    if (currentMillis - previousRadarTime >= 100) {
```

```
    ESPSerial.print('L');  
    previousRadarTime = currentMillis;  
}  
if (currentMillis - phaseTimer >= 150) {  
    patrolPhase = 4; phaseTimer = currentMillis;  
}  
}  
else if (patrolPhase == 4) {  
    if (currentMillis - previousRadarTime >= 100) {  
        ESPSerial.print('R');  
        previousRadarTime = currentMillis;  
    }  
    if (currentMillis - phaseTimer >= 800) {  
        patrolPhase = 0; phaseTimer = currentMillis;  
    }  
}  
}  
}
```

YOLO AI

```
import cv2  
import sys  
import time  
import requests
```

```

import numpy as np

from ultralytics import YOLO

model = YOLO('yolo11n.pt')

ESP32_IP = "192.168.4.1"

ESP32_CAPTURE_URL = f"http://{ESP32_IP}:80/capture"

ESP32_ACTION_URL = f"http://{ESP32_IP}/action?cmd="

print("Connecting to ESP32-CAM via Capture Stream...")

current_state = "S"

last_printed_state = ""

is_fullscreen = False

yolo_auto_mode = False

cycle_start_time = time.time()

# Smart Turn Variables

turn_delay_active = False

turn_start_time = 0

pending_turn_cmd = ""

def send_bot_command(cmd, description):

    global last_printed_state

    if cmd != last_printed_state:

        print(f">>> BOT COMMAND: {description} <<<")

        last_printed_state = cmd

    try:

        requests.get(ESP32_ACTION_URL + cmd, timeout=0.2)

    except requests.exceptions.RequestException:

        pass

window_name = "ESP32-CAM YOLO Navigation"

```

```

cv2.namedWindow(window_name, cv2.WINDOW_NORMAL)

print("\n--- CONTROLS ---")

print("Press 'A' to enable YOLO Auto Mode (moves autonomously)")

print("Press 'M' to enable Manual/Pause Mode (stops bot, use Web UI to drive)")

print("Press 'Q' to Quit")

print("-----\n")

while True:

    try:

        img_resp = requests.get(ESP32_CAPTURE_URL, timeout=1.0)

        img_arr = np.array(bytearray(img_resp.content), dtype=np.uint8)

        frame = cv2.imdecode(img_arr, -1)

        if frame is None:

            continue

    except Exception as e:

        time.sleep(0.5)

        continue

    height, width, _ = frame.shape

    left_zone_end = width // 3

    right_zone_start = (width // 3) * 2

    results = model(frame, conf=0.4, stream=True, verbose=False)

    obstacle_in_path = False

    left_side_objects = 0

    right_side_objects = 0

    for r in results:

        for box in r.bboxes:

            x1, y1, x2, y2 = map(int, box.xyxy[0])

```

```

conf = box.conf[0]

cls = int(box.cls[0])

class_name = model.names[cls]

center_x = (x1 + x2) // 2

if left_zone_end < center_x < right_zone_start:

    obstacle_in_path = True

    box_color = (0, 0, 255)

else:

    box_color = (0, 255, 0)

cv2.rectangle(frame, (x1, y1), (x2, y2), box_color, 2)

cv2.putText(frame, f"{class_name}{conf:.2f}", (x1, y1 - 10),

            cv2.FONT_HERSHEY_SIMPLEX, 0.5, box_color, 2)

if center_x < width // 2:

    left_side_objects += 1

else:

    right_side_objects += 1

cv2.line(frame, (left_zone_end, 0), (left_zone_end, height), (255, 0, 0), 2)

cv2.line(frame, (right_zone_start, 0), (right_zone_start, height), (255, 0, 0), 2)

current_time = time.time()

# --- MODE & CYCLE LOGIC ---

if not yolo_auto_mode:

    cv2.putText(frame, "MODE: MANUAL/PAUSED (Press 'A' for Auto)", (10, 30),

cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 255), 2)

else:

    # Auto Mode

    cv2.putText(frame, "MODE: YOLO AUTO (Press 'M' for Manual)", (10, 30),

cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

```

```

# 5s Move, 2s Brake Timer Logic

cycle_elapsed = current_time - cycle_start_time

if cycle_elapsed >= 7.0:

    cycle_start_time = current_time

    cycle_elapsed = 0.0

if cycle_elapsed >= 5.0:

    send_bot_command('S', "CYCLE BRAKE (2s)")

    cv2.putText(frame, "CYCLE BRAKE (2s)...", (50, 80), cv2.FONT_HERSHEY_SIMPLEX, 1,
(0, 165, 255), 3)

else:

    if obstacle_in_path:

        if not turn_delay_active:

            send_bot_command('S', "OBSTACLE! BRAKING & SCANNING...")

            cv2.putText(frame, "SCANNING L/R...", (50, 80), cv2.FONT_HERSHEY_SIMPLEX, 1,
(0, 0, 255), 3)

            turn_delay_active = True

            turn_start_time = current_time

            if left_side_objects == 0 and right_side_objects > 0:

                pending_turn_cmd = 'L'

            elif right_side_objects == 0 and left_side_objects > 0:

                pending_turn_cmd = 'R'

            else:

                pending_turn_cmd = 'L' if left_side_objects <= right_side_objects else 'R'

        elif current_time - turn_start_time >= 0.8:

            if pending_turn_cmd == 'L':

                send_bot_command('L', "LEFT IS CLEAR. TURNING LEFT")

```

```

        cv2.putText(frame, "TURNING LEFT", (50, 80), cv2.FONT_HERSHEY_SIMPLEX, 1,
(0, 165, 255), 3)

    else:

        send_bot_command('R', "RIGHT IS CLEAR. TURNING RIGHT")

        cv2.putText(frame, "TURNING RIGHT", (50, 80), cv2.FONT_HERSHEY_SIMPLEX,
1, (0, 165, 255), 3)

    else:

        turn_delay_active = False

        send_bot_command('F', "PATH CLEAR. MOVE FORWARD")

        cv2.putText(frame, "FORWARD", (50, 80), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255,
0), 3)

cv2.imshow(window_name, frame)

# --- Keyboard Listening ---

key = cv2.waitKey(1) & 0xFF

if key == ord('q'):

    send_bot_command('S', "STOPPING BOT AND EXITING")

    break

elif key == ord('a'): # Enable Auto

    yolo_auto_mode = True

    cycle_start_time = time.time()

    last_printed_state = ""

    print("Switched to YOLO AUTO mode.")

    try: requests.get(f"http://{ESP32_IP}/mode?val=Y", timeout=0.2)

    except: pass

elif key == ord('m'): # Enable Manual/Pause

    yolo_auto_mode = False

    last_printed_state = ""

```

```
send_bot_command('S', "MANUAL MODE ACTIVATED")

print("Switched to MANUAL mode.")

try: requests.get(f"http://{ESP32_IP}/mode?val=M", timeout=0.2)
except: pass

elif key == ord('f'):
    is_fullscreen = not is_fullscreen

    if is_fullscreen:
        cv2.setWindowProperty(window_name, cv2.WND_PROP_FULLSCREEN,
cv2.WINDOW_FULLSCREEN)

    else:
        cv2.setWindowProperty(window_name, cv2.WND_PROP_FULLSCREEN,
cv2.WINDOW_NORMAL)

cv2.destroyAllWindows()
```